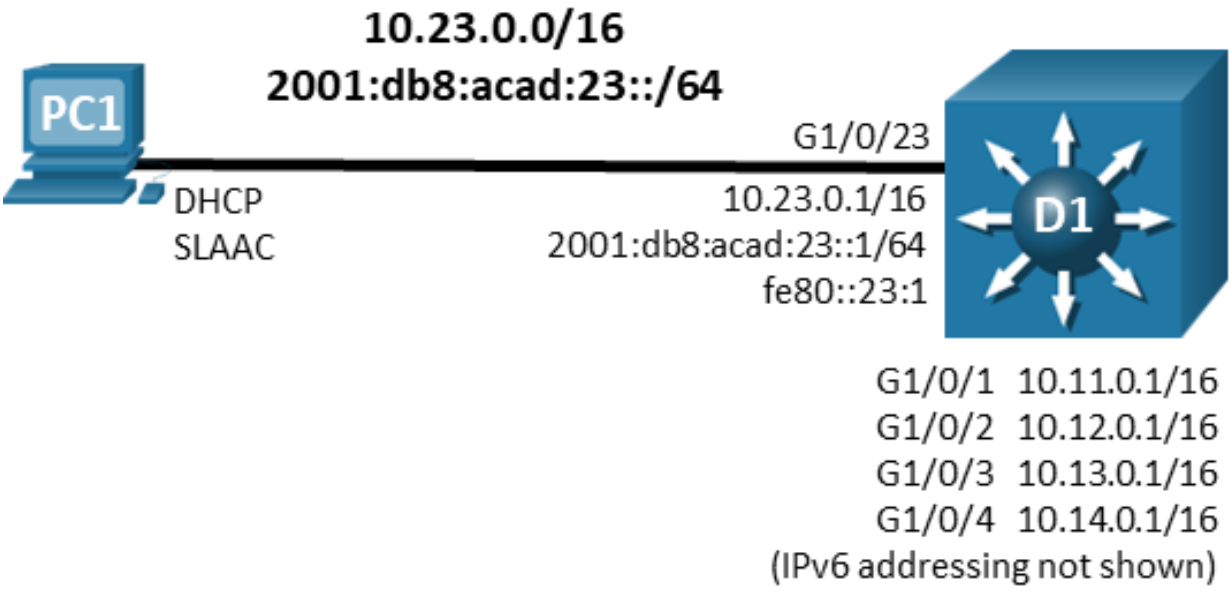
Lab - Use RESTCONF to Access an IOS XE L3 Switch

Topology



Addressing Table

Device	Interface	IP Address	Subnet Mask
D1	G1/0/23	10.23.0.1	255.255.0.0
PC	NIC	DHCP	DHCP

Objectives

Part 1: Configure an IOS XE Device for RESTCONF Access

Part 2: Open and Configure Postman

Part 3: Use Postman to Send GET Requests

Part 4: Use RESTCONF and YANG to gather structured, agnostic device information

Part 5: Use Postman to Send a PUT Request

Part 6: Use interfaces-state to gather interface state information

Part 7: Use a Python script to Send GET Requests

Part 8: Use a Python Script to Send a PUT Request

Part 9: Real-World Example — Using Python and RESTCONF for Interface Monitoring

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Background / Scenario

The RESTCONF protocol is a subset of NETCONF. RESTCONF allows you to make RESTful API calls to an IOS XE device. The data that is returned by the API can be either XML or JSON. In the first parts of this lab, you will use the Postman program to construct and send API requests to the RESTCONF service that is running on D1. In the latter parts of the lab, you will create Python scripts to perform the same tasks as your Postman program.

Required Resources

- 1 Switch (Cisco 9300L with Cisco IOS XE Release 17.12.5 universal image or comparable)
- 1 PCs (Choice of operating system with terminal emulation program installed and **Postman installed**)
- Console cables to configure the Cisco IOS devices via the console ports
- Ethernet cables as shown in the topology

Instructions

Part 1: Configure an IOS XE Device for RESTCONF Access

In Part 1, you will configure D1 to accept RESTCONF messages. You will also start the HTTPS service on D1.

Step 1: Cable the network as shown in the topology.

Attach the devices as shown in the topology diagram and cable as necessary.

Step 2: Configure basic settings for each device.

a. Console into D1, enter global configuration mode, and apply the following basic settings.

Switch D1

```
hostname D1
no ip domain lookup
ip routing
ipv6 unicast-routing
banner motd # D1, Use RESTCONF to Access an IOS XE Device #
line con 0
 exec-timeout 0 0
 logging synchronous
 exit
line vty 0 15
 privilege level 15
 logging synchronous
 exec-timeout 0 0
 transport input ssh
 login local
 exit
enable secret cisco12345
ip domain name example.netacad.com
username admin privilege 15 secret cisco123!
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
interface range g1/0/1-24, g1/1/1-4, g0/0
 shutdown
interface GigabitEthernet1/0/1
 description Link to Sales network
 no switchport
 ip address 10.11.0.1 255.255.0.0
 ipv6 address 2001:db8:acad:11::1/64
 no shutdown
 exit
interface GigabitEthernet1/0/2
 description Link to Engineering network
 no switchport
 ip address 10.12.0.1 255.255.0.0
 ipv6 address 2001:db8:acad:12::1/64
 no shutdown
 exit
interface GigabitEthernet1/0/3
 description Link to HR network
 no switchport
 ip address 10.13.0.1 255.255.0.0
 ipv6 address 2001:db8:acad:13::1/64
 no shutdown
 exit
interface GigabitEthernet1/0/4
 description Link to DC network
 no switchport
 ip address 10.14.0.1 255.255.0.0
 ipv6 address 2001:db8:acad:14::1/64
 no shutdown
 exit
interface GigabitEthernet1/0/23
 description Link to PC
 no switchport
 ip address 10.23.0.1 255.255.0.0
 ipv6 address fe80::23:1 link-local
 ipv6 address 2001:db8:acad:23::1/64
 no shutdown
 exit
ip domain name example.netacad.com
ip dhcp excluded-address 10.23.0.1 10.23.0.9
ip dhcp pool LAN
 network 10.23.0.0 255.255.0.0
 default-router 10.23.0.1
 domain-name example.netacad.com
crypto key generate rsa modulus 2048
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
end
```

b. Set the clock on D1 to UTC time.

c. Save the running configuration to startup-config.

d. Verify that PC1 can ping D1's IPv4 and IPv6 address. If not, troubleshoot until successful.

e. Verify that you can access D1 using an SSH connection from PC1.

Step 3: Verify that RESTCONF is running on D1.

RESTCONF may already be running if another student enabled it, or if a later IOS version enables it by default. From the PuTTY terminal, you can use the **show platform software yang-management process** command to display the status of all YANG-related management processes, including those required for RESTCONF and NETCONF. It helps verify which daemons are currently running and ensures the RESTCONF service is operational. A **daemon** is a background process that runs continuously on a device or operating system to perform specific system or service tasks without direct user interaction.

```
D1# show platform software yang-management process
confd            : Not Running
nesd             : Not Running
syncfd           : Not Running
ncsshd           : Not Running
dmiauthd         : Not Running
nginx            : Not Running
ndbmand          : Not Running
pubd             : Not Running
gnmib            : Not Running
```

For RESTCONF to function properly on an IOS XE device, the processes highlighted above must be **running**:

- confd
- nesd
- syncfd
- dmiauthd
- nginx
- ndbmand

Note: The **nginx** process is the HTTPS web server that handles all RESTCONF connections over port 443. The remaining daemons synchronize configuration and operational data between IOS XE and the YANG database. The **ncsshd**, **pubd**, and **gnmib** processes are related to NETCONF or model-driven telemetry and are not required for RESTCONF.

The specific function of all the daemons is not important; however, the list below provides an overview of each process.

- **confd** - Core configuration daemon that manages YANG-modeled configuration data and provides the RESTCONF/NETCONF interface to it.
- **nesd** - NETCONF/RESTCONF southbound daemon that synchronizes configuration data between IOS XE and the YANG data store.
- **syncfd** -Keeps the running configuration on the device synchronized with the YANG database (so changes in IOS CLI or API stay consistent).

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

- **ncsshd** - NETCONF Secure Shell daemon that handles NETCONF connections over SSH; not used for RESTCONF but often runs alongside it.
- **dmiauthd** - Device-Management Interface authentication daemon that validates user credentials for NETCONF and RESTCONF sessions.
- **nginx** -Embedded HTTPS web server that carries all RESTCONF traffic; it must be running for RESTCONF over HTTPS to work.
- **ndbmand** - Network Database Management daemon that maintains the operational (state) YANG data for features such as interfaces-state, routing, etc.
- **pubd** - Publisher daemon that pushes YANG event notifications or telemetry data to subscribers (used for model-driven telemetry).
- **gnmib** - gNMI/gRPC interface daemon providing an alternative API for streaming telemetry and configuration using OpenConfig models.

Step 4: Enable and verify the RESTCONF service.

a. Enter the global configuration command **restconf** to enable the RESTCONF service on D1.

```
D1(config)# restconf
```

b. You will see several IOS XE messages confirming that the RESTCONF-related processes are starting. These messages indicate that the RESTCONF service and its supporting daemons have been initialized. Example messages may include:

```
%PSD_MOD-5-DMI_NOTIFY_RESTCONF_START: Switch 1 R0/0: psd: PSD/DMI: restconf server has been notified to start
%NDBMAN-5-ACTIVE: Switch 1 R0/0: ndbmand: All data providers active.
%DMI-5-NACM_INIT: Switch 1 R0/0: dmiauthd: NACM configuration has been set to its initial configuration.
```

Note: These messages confirm that the RESTCONF environment is being brought online and to ensure that the required YANG management processes are launching successfully. Messages may come in slowly. You can proceed if you haven't seen all the examples shown above.

c. Verify that the required RESTCONF daemons are now running.

The **restconf** command starts the YANG management processes needed for the RESTCONF API. Note that **ncsshd** (for NETCONF) and other optional services may appear as *Running* or *Not Running* depending on the device and version. These are not required for this lab, but **nginx** (the HTTPS server) must eventually be *Running* for RESTCONF to work.

```
D1# show platform software yang-management process
confd            : Running
nesd             : Running
syncfd           : Running
ncsshd           : Not Running
dmiauthd         : Running
nginx            : Not Running
ndbmand          : Running
pubd             : Running
gnmib            : Not Running
```

Step 5: Enable and verify the HTTPS service.

a. Enter the following global configuration commands to enable the HTTPS server and specify that user authentication will use the local database.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
D1# configure terminal
D1(config)# ip http secure-server
D1(config)# ip http authentication local
D1(config)# exit
```

Note: Even if the **nginx** process already shows as *Running*, enter these commands to make sure the HTTPS service and local authentication are correctly configured for RESTCONF. It is best to re-enter them.

b. Verify that the HTTPS server (nginx) is now running.

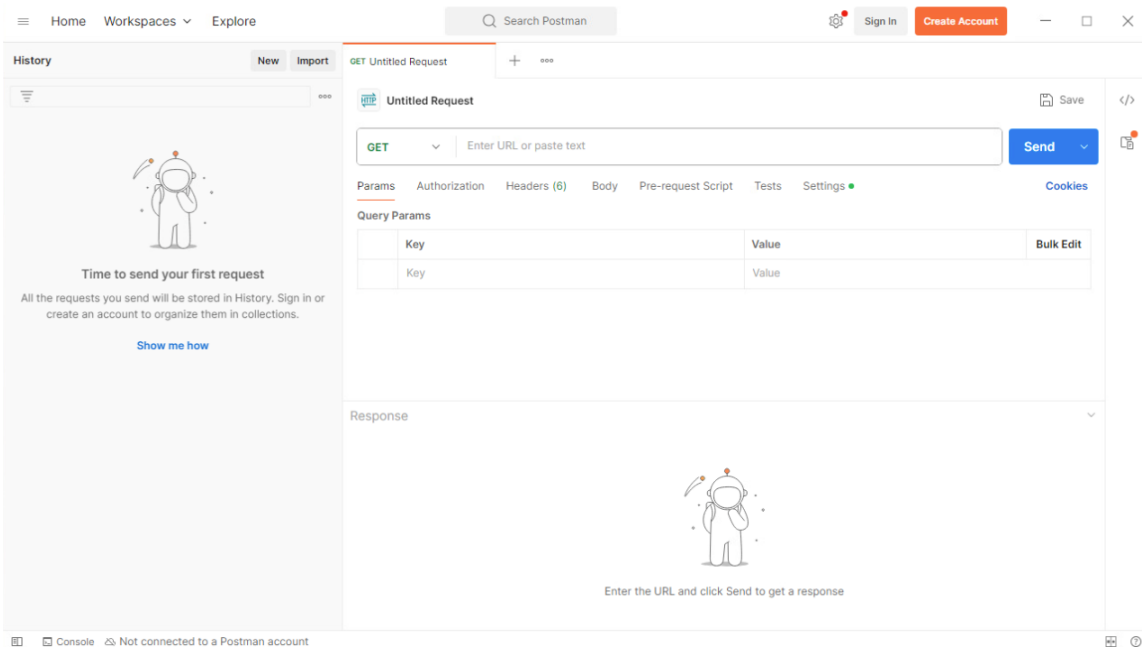
```
D1# show platform software yang-management process
confd            : Running
nesd             : Running
syncfd           : Running
ncsshd           : Not Running
dmiauthd         : Running
nginx            : Running
ndbmand          : Running
pubd             : Running
gnmib            : Not Running
```

At this point, RESTCONF and HTTPS are both active on D1, and the device is ready to accept REST API requests securely over port 443 using the **nginx** HTTPS server.

Part 2: Open and Configure Postman

In Part 2, you will open Postman, disable SSL certificates, and explore the user interface.

Step 1: Open Postman.



a. When you first open the Postman application, you should see a **new request window** in the main area.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

- The **method selector** (set to **GET** by default) appears to the left of the **Enter URL or paste text** field, where you'll type the RESTCONF URL.
- Below the URL field are several **tabs** labeled **Params**, **Authorization**, **Headers**, **Body**, **Pre-request Script**, **Tests**, and **Settings**.
- On the left side, you'll see the **History** panel, which will list any requests you make during this lab.
- At the bottom of the window, you'll find the **Response** area where Postman will display the data returned by your request.

b. If Postman asks you to sign in or create an account, you can click **"Skip and continue without an account."** Signing in is not required for this lab.

Step 2: Disable SSL certification verification.

By default, Postman has SSL certification verification turned on. You will not be using SSL certificates with D1; therefore, you need to turn off this feature.

a. Click the **Settings** tab.

b. Set the **Enable SSL certificate verification** to **OFF**.

Step 3: Explore the Postman interface Tabs.

Postman organizes requests into several tabs that help you build and test REST API calls. Take a moment to click through each of the main tabs below to become familiar with them before sending your first request.

- **Params**: Used to add query parameters to the request URL.
- **Authorization**: Used to provide login credentials or authentication tokens. You'll use **Basic Auth** in this lab.
- **Headers**: Defines the format of the data sent to and received from the device (for example, JSON).
- **Body**: Used to include data when sending configuration changes such as POST or PUT requests.
- **Pre-request Script** and **Tests**: Advanced options for automated scripting or testing. You won't need these in this lab, but it's useful to know they exist.

Part 3: Use Postman to Send GET Requests

In Part 3, you will use Postman to send a GET request to D1 to verify that you can connect to the RESTCONF service.

Step 1: Enter the URL for a D1.

a. Confirm that the request type is already set to **GET**.

b. In the **Enter request URL** field, type the URL used to reach the RESTCONF root on D1:

```
https://10.23.0.1/restconf/
```

This verifies basic RESTCONF connectivity.

Note: When accessing a RESTCONF API over IPv6, the IPv6 address must be enclosed in square brackets ([ ]) in the URL. This is required because IPv6 addresses contain colons (:), and the brackets prevent them from being confused with the colon that separates the host from the **TCP port number** in a URL. For example:

```
https://[2001:db8:acad:23::1]/restconf/
```

Step 2: Enter authentication credentials.

In this lab, you'll mainly use the **Authorization** and **Headers** tabs.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

a. Click the **Authorization** tab.

b. From the **Type** drop-down menu (it may show *No Auth* or *Inherit auth from parent*), choose **Basic Auth**.

c. Enter the login credentials that you configured on **D1** in Part 1:

Username: **admin**

Password: **cisco123!**

Step 3: Verify that the authentication header is created automatically.

When you choose **Basic Auth** and enter your credentials, Postman automatically generates the required **Authorization** header in the background. This header carries your credentials (encoded, not encrypted — encryption is handled by HTTPS) so D1 can verify your identity. The **Authorization** header is required by RESTCONF and most other HTTP-based APIs to authenticate the request.

a. Click the **Headers** tab.

b. If some headers are hidden, click the number of **hidden** headers, such as **7 hidden**, to expand the list. You may need to expand or scroll the window to see everything.

c. You should see an entry similar to:

Key: Authorization Value: Basic Y2lzY286Y2lzY28xMjMh

The long string is the Base64-encoded version of your username and password.

Step 4: Set JSON as the data type to send to and receive from D1.

You can send and receive data from D1 in either XML or JSON format. In this lab, you will use **JSON**, which represents data as key–value pairs that closely match Python dictionaries.

In the **Headers** tab, add a new key-value pair below **Connection**.

a. Click in the **Key** field and type **Accept**.

In the **Value** field, type **application/yang-data+json**.

This tells D1 that you want the RESTCONF response returned in JSON format.

Note: When you type in the **Key** or **Value** fields, Postman may display a drop-down list of common options. The fields you need for this lab, especially the value application/yang-data+json, may not appear in the list, but you can still type them manually, and Postman will accept them.

b. Add another key-value pair for the data you are sending:

1) Click in the **Key** field and type **Content-Type**.

2) In the Value field, type **application/yang-data+json**.

Even though GET requests don't include a body, this header helps reinforce the format for later PUT and POST requests.

Make sure both checkboxes next to your new headers are **checked** so Postman includes them when you send the request.

Note: You can change **application/yang-data+json** to **application/yang-data+xml** to send and receive XML data instead of JSON data, if necessary.

Step 5: Send the API request to D1.

Postman now has all the information it needs to send the GET request.

Click **Send**.

If RESTCONF is configured correctly, you should see a **Status: 200 OK** message next to the URL field and a formatted JSON response in the **Body** section, such as:

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
{
    "ietf-restconf:restconf": {
        "data": {},
        "operations": {},
        "yang-library-version": "2019-01-04"
    }
}
```

This confirms that D1 is responding to RESTCONF requests and that Postman can now retrieve YANG-modeled data.

Exploring the Response View

- The **Pretty** tab displays the JSON data in an easy-to-read, indented format.
- The **Raw** tab shows the unformatted data exactly as it was returned by the device.
- The **Status 200 OK** indicates a successful request. Postman successfully contacted D1 and received valid data.

Troubleshooting Tips

If you don't receive a 200 OK response:

- **401 Unauthorized**: Check that your username, password, or **ip http authentication local** is correctly configured.
- **404 Not Found**: The RESTCONF path or YANG model name may be incorrect. Check spelling, colons, and capitalization.
- **406 / 415 Errors**: Your **Accept** or **Content-Type** headers may be missing or incorrect.
- **SSL or certificate warnings**: Ensure **SSL certificate verification** is disabled in Postman for this lab.

Part 4: Use RESTCONF and YANG to gather structured, agnostic device information

In this part, you'll look more closely at how RESTCONF uses YANG models to provide structured, consistent information from a network device. Instead of relying on traditional CLI commands, where the format varies by vendor, platform, and even software version, RESTCONF returns data in a predictable, machine-readable structure defined by YANG. This allows tools like Python, Postman, and automation frameworks to retrieve and analyze device information in a vendor-agnostic way, making automation easier, more reliable, and far more scalable.

Step 1: Use a GET request to gather the information for all interfaces on D1.

a. Now that you have a successful GET request, you can use it as a template for additional requests.

   At the top of Postman, next to the **Launchpad** tab, right-click the **GET** tab that you just used and choose **Duplicate Tab**.

b. Use the **ietf-interfaces** YANG model to gather interface information. In the URL field, enter:

   ```
   https://10.23.0.1/restconf/data/ietf-interfaces:interfaces
   ```

   Note: If you receive a 403 Forbidden error, check your **username** configuration. Make sure it includes the **privilege 15** argument.

c. Because you copied your previous **GET** request, the **Authorization** and **Header** data is already populated. Click **Send**. You should see a JSON response from D1 that is similar to the output below. Your output may vary depending on your switch model.

   ```
   {
   ```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
"ietf-interfaces:interfaces": {
  "interface": [
    {
      "name": "GigabitEthernet0/0",
      "type": "iana-if-type:ethernetCsmacd",
      "enabled": false,
      "ietf-ip:ipv4": {},
      "ietf-ip:ipv6": {}
    },
    {
      "name": "GigabitEthernet1/0/1",
      "description": "Link to Sales network",
      "type": "iana-if-type:ethernetCsmacd",
      "enabled": true,
      "ietf-ip:ipv4": {
          "address": [
              {
                  "ip": "10.11.0.1",
                  "netmask": "255.255.0.0"
              }
          ]
      },
      "ietf-ip:ipv6": [
          "address": [
              {
                  "ip": "2001:db8:acad:11::1",
                  "prefix-length": 64
              }
          ]
      },
    <output omitted>
    {
      "name": "GigabitEthernet1/0/23",
      "description": "Link to PC",
      "type": "iana-if-type:ethernetCsmacd",
      "enabled": true,
      "ietf-ip:ipv4": {
          "address": [
              {
                  "ip": "10.23.0.1",
                  "netmask": "255.255.0.0"
              }
          ]
      },
      "ietf-ip:ipv6": [
          "address": [
              {
                  "ip": "2001:db8:acad:23::1",
                  "prefix-length": 64
```


© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
                    }
                ]
            },
            <output omitted>
            }
        ]
    }
}
```

Step 2: Understand structured data and key–value pairs.

Key-Value Pairs

The JSON output returned by RESTCONF is a form of **structured data**. The information is organized into predictable patterns that both humans and programs can easily read and process.

Each piece of data is stored as a **key–value pair**, written in the format:

```
"key": "value"
```

For example:

```
"name": "GigabitEthernet1/0/23"
```

In this example, **name** is the key (the label), and **GigabitEthernet1/0/23** is the value. Keys always appear in quotes, followed by a colon and their value. Values can be different data types. For example, **strings** (in quotes), **numbers** (integers or decimals, written without quotes), or **Booleans** (true or false, also without quotes).

JSON and Python Dictionaries

JSON (JavaScript Object Notation) and Python **dictionaries** look almost identical. Both use curly braces { } and store data as key–value pairs.

One difference is JSON requires **double quotes** around all keys and string values. However, Python allows either **single** or **double quotes**, so both of the following are valid in Python.

We will discuss JSON further in Part 7 when we use Python code to send our RESTCONF requests.

Step 3: Understand YANG models.

A **YANG model** is a structured blueprint that defines how data on a network device is organized and accessed. Using YANG models provides a consistent, standardized way to retrieve and configure data across different devices, vendors and APIs, making automation more reliable and predictable.

There are two types of YANG models:

- **IETF models**: These models start with **ietf-** (such as **ietf-interfaces**) and are **standards-based**, defined by the Internet Engineering Task Force (IETF). Other example include: **ietf-ip** and **ietf-routing**. These models describe common, vendor-neutral data like IP addressing or routing.
- **Cisco native models**: These models start with **Cisco-IOS-** and are Cisco-specific extensions that include features unique to IOS XE. Cisco IOS XE examples include **Cisco-IOS-XE-vlan-oper** and **Cisco-IOS-XE-native**. These models describe things like VLANs, EEM, QoS, and other platform-specific capabilities.

Note: You can explore any model and its tree structure at yangcatalog.org.

Advantages of using YANG models:

- Produce the **same data structure and field names** across different devices and vendors.
- Make **automation and scripting easier**, because tools can expect consistent output.
- Allow **data to be parsed and analyzed** programmatically for reporting and monitoring.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

- Enable **interoperability** between management tools, controllers, and APIs (NETCONF, RESTCONF).

Step 4: View all supported YANG models on D1.

You can use RESTCONF to retrieve a list of all YANG models supported by this device. Each model describes a specific set of data or configuration options available through the API.

Duplicate your current tab, and then enter the following **GET** request in Postman:

```
https://10.23.0.1/restconf/data/ietf-yang-library:modules-state/module
```

This request returns a list of every YANG module implemented on D1, including both **IETF standard models** (such as **ietf-interfaces**) and **Cisco-specific models** (such as **Cisco-IOS-XE-vlan-oper**).

The output includes details like the module's name, namespace, and revision. Viewing the list of models helps you understand what information and features are accessible through RESTCONF. It's the device's "menu" of supported APIs.

Note: To access a YANG model through RESTCONF, you must convert the **module name** from the list into a RESTCONF URL, which is beyond the scope of this lab. However, you can look up the correct URL format and top-level container for any module by searching for it on the **Cisco DevNet YANG Browser** (https://yangcatalog.org) or the **Cisco YANG Models Repository** (https://github.com/YangModels/yang).

Step 5: Use a GET request to gather information for a specific interface on D1.

To view data for one specific interface, you can extend the URL to request only that interface's information.

a. Duplicate the previous **ietf-interfaces GET** request tab in Postman.

b. Add the **interface=** parameter to specify the exact **interface name**.

   However, you must replace each forward slash / in the interface name with its URL encoded value **%2F.**

   For example,1/0/23 becomes 1**%2F**0**%2F**23.

```
https://10.23.0.1/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1%2F0%2F23
```

   Note: The above URL should be on one line. The YANG model we are using is **ietf-interfaces.**

c. Click **Send**. You should see a JSON response from D1 similar to the example below. Your output may vary depending on your device's configuration


```
{
  "ietf-interfaces:interfaces": [
      {
        "name": "GigabitEthernet1/0/23",
        "description": "Link to PC",
        "type": "iana-if-type:ethernetCsmacd",
        "enabled": true,
        "ietf-ip:ipv4": {
            "address": [
                {
                    "ip": "10.23.0.1",
                    "netmask": "255.255.0.0"
                }
            ]
        },
        "ietf-ip:ipv6": [
            "address": [
                {
                    "ip": "2001:db8:acad:23::1",
```


© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
                        "prefix-length": 64
                    }
                ]
            }
        }
    ]
}
```

Note: **Copy** the output to a text file. We will use this information in Part 5.

Part 5: Use Postman to Send a PUT Request

In Part 5, you will configure Postman to send a PUT request to D1 to create a new loopback interface.

Step 1: Duplicate and modify the last GET request.

Note: For this step, use the Loopback interface information assigned by your instructor. If no assignment was given and no other students are using your D1 device, you can use the example below.

a. Duplicate your last GET request.

b. Change the request **Type** from **GET** to **PUT** (using the dropdown menu next to the URL).

c. Modify the URL to specify the new Loopback interface:

`https://10.23.0.1/restconf/data/ietf-interfaces:interfaces/interface=Loopback1`

Step 2: Configure the body of the request specifying the information for the new loopback.

a. Click the **Body** tab (top section), then select **raw**, and set the type to **JSON**.

If you click **Send** with an empty body, you'll receive a 400 Bad Request error because RESTCONF requires the full interface definition to create the resource.

Note: Make sure you're in the *request* Body tab (**top**), not the *response* Body (bottom).

b. Enter the following JSON in the Body field. Replace [Your Name] with your own name (without brackets) in the description field.

Notice that the type of interface must be set to **softwareLoopback**.

```
{
  "ietf-interfaces:interface": {
    "name": "Loopback1",
    "description": "[Your Name] Loopback",
    "type": "iana-if-type:softwareLoopback",
    "enabled": true,
    "ietf-ip:ipv4": {
      "address": [
        {
          "ip": "10.100.0.1",
          "netmask": "255.255.0.0"
        }
      ]
    }
  }
}
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Alternate method: Edit your previous GET response

If you prefer, you can reuse the data from your previous **GET** request that you save to a text file. You can edit this to create the PUT body:

- Copy the **JSON data** returned from your last GET request (everything inside the curly braces { ... }).
- Keep only **one interface object** — delete any extra interfaces from the list.
- Remove the outer wrapper "ietf-interfaces:interfaces": [and its closing] and replace it with "ietf-interfaces:interface": { at the top.
- Update the "name" field to the new interface name (for example, Loopback1).
- Change the "type" to iana-if-type:softwareLoopback.
- Modify the "description" to include your name.
- Update the IPv4 "ip" and "netmask" values as assigned.
- Delete the "ietf-ip:ipv6" section.
- Check that **all braces and brackets are balanced**, and **no commas** follow the last item in a list. Be careful, some are brackets!
- Paste the finished JSON into Postman's **Body** → **raw** → **JSON** editor.

Note: The final structure should start with "ietf-interfaces:interface" (singular), not "ietf-interfaces:interfaces" (plural).

Step 3: Send and Verify.

a. Click **Send** to send the PUT request.

A successful response will return either:

- 201 Created: The interface was created.
- 204 No Content: The configuration was accepted (depending on IOS XE version).

b. Verify that the interface was created by entering **show ip interface brief** on D1. Any interface listed with an **OK?** value of **NO** does not have a valid configuration.

```
D1# show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  administratively down down
<output omitted>
Loopback1              10.100.1.1      YES other  up                    up
```

Note: You can also use the Postman request you created in **Part 4, Step 1** to verify that the new Loopback interface appears along with the existing interfaces. Example:

```
https://10.23.0.1/restconf/data/ietf-interfaces:interfaces
```

Part 6: Use interfaces-state to gather interface state information

The **ietf-interfaces:interfaces-state** YANG model provides *operational* (real-time) interface data from the device.

Unlike the **ietf-interfaces:interfaces** model, which shows the *configured* interfaces and their settings, **interfaces-state** reports the current link status, input/output counters, error statistics, and protocol state (up/down). This allows you to verify whether interfaces are active and transmitting correctly, making it useful for network monitoring and troubleshooting.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

You will also see more examples of how device information can be displayed as structure data using key-value pairs.

Step 1: Duplicate and modify the previous GET request.

a. Duplicate your last **GET** request.

b. Enter or modify the previous URL to specify the new URL:

```
https://10.23.0.1/restconf/data/ietf-interfaces:interfaces-state
```

Step 2: View and interpret interface state information.

a. Click **Send**. You should see a JSON response from D1 that is similar to the output below. Your output may vary depending on your switch model and which interfaces are active.

Most of these interfaces will not contain any information. Scroll down until you see GigabitEthernet1/0/23. This is the interface which is connected to PC1.

```
{
    "ietf-interfaces:interfaces-state": {
        "interface": [
            {
                "name": "GigabitEthernet1/0/1",
                <output omitted>
            },
            {
                "name": "GigabitEthernet1/0/23",
                "type": "iana-if-type:ethernetCsmacd",
                "admin-status": "up",
                "oper-status": "up",
                "last-change": "2025-11-13T15:43:29.586+00:00",
                "if-index": 31,
                "phys-address": "f8:e9:4f:9b:7f:e7",
                "speed": "1000000000",
                "statistics": {
                    "discontinuity-time": "2025-11-13T14:49:03.586+00:00",
                    "in-octets": "1515334",
                    "in-unicast-pkts": "13361",
                    "in-broadcast-pkts": "522",
                    "in-multicast-pkts": "1176",
                    "in-discards": 0,
                    "in-errors": 0,
                    "in-unknown-protos": 0,
                    "out-octets": "1031404",
                    "out-unicast-pkts": "4320",
                    "out-broadcast-pkts": "693",
                    "out-multicast-pkts": "2561",
                    "out-discards": 0,
                    "out-errors": 0
                }
            }
            <output omitted>
            {
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
                "name": "vlan1",
                <output omitted>
            }
        ]
    }
}
```


The **ietf-interfaces:interfaces-state** data shows *operational* statistics for each interface. Each interface object includes key fields such as:

- **name**: The interface identifier (for example, *GigabitEthernet1/0/23*).
- **admin-status**: Whether the interface has been administratively enabled or disabled.
- **oper-status**: Whether the interface is actually up or down (reflects link state).
- **last-change**: The exact time the interface last changed its operational state, such as going up, down, or being reset after a reload.
- **phys-address**: The interface MAC address.
- **speed**: Link bandwidth in bits per second.
- **statistics**: Counters for bytes and packets sent or received, including unicast, multicast, broadcast, errors, and discards.
- **discontinuity-time**: The last time the interface counters were reset or cleared (such as after a reboot or interface flap), marking when the current statistics began accumulating.

These fields let you confirm both the configuration and the real-time health of the interface, which is similar to the IOS **show interfaces** or **show interfaces status** commands, but returned in structured JSON format for use in automation or monitoring tools.

Step 3: Filter for a specific interface (PC connection).

a. Duplicate your **GET** request in the previous step.

b. Modify the URL so that it retrieves information only for the interface connected to PC1, **GigabitEthernet1/0/23**.

```
https://10.23.0.1/restconf/data/ietf-interfaces:interfaces-state/interface=GigabitEthernet1%2F0%2F23
```

Note: The interface name must match the actual IOS XE interface name (as shown in **show run** or **show ip interface brief**).

c. Click **Send**. You should see a JSON response from D1 that is similar to the output below. Your output may vary depending on your switch model and which interfaces are active.

The output displayed with be key-value pairs specific to the GigabitEthernet1/0/23 interface.


```
{
    "ietf-interfaces:interfaces-state": {
        "interface": [
            {
                "name": "GigabitEthernet1/0/23",
                "type": "iana-if-type:ethernetCsmacd",
                "admin-status": "up",
                "oper-status": "up",
                "last-change": "2025-11-13T15:43:29.586+00:00",
                "if-index": 31,
                "phys-address": "f8:e9:4f:9b:7f:e7",
```


© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
                "speed": "1000000000",
                "statistics": {
                    "discontinuity-time": "2025-11-13T14:49:03.586+00:00",
                    "in-octets": "2257301",
                    "in-unicast-pkts": "20340",
                    "in-broadcast-pkts": "622",
                    "in-multicast-pkts": "1176",
                    "in-discards": 0,
                    "in-errors": 0,
                    "in-unknown-protos": 0,
                    "out-octets": "1705206",
                    "out-unicast-pkts": "8698",
                    "out-broadcast-pkts": "693",
                    "out-multicast-pkts": "2657",
                    "out-discards": 0,
                    "out-errors": 0
                }
            }
        ]
    }
}
```


When you resend the URL for the specific interface, the values under **statistics** (such as in-octets, out-octets, and packet counters) will likely increase as traffic flows, except for **discontinuity-time** which will remain unchanged unless the interface counters are cleared or the device is reloaded.

Part 7: Use a Python script to Send GET Requests

In Part 7, you will create a Python script to send GET requests to D1.

Step 1: Create the Python program for a GET Request.

a. Open IDLE, VS Code, or another Python IDE.

b. Create a new Python file and save the file as **restconf-get.py**.

c. Enter the following Python code:

```
import json
import requests
requests.packages.urllib3.disable_warnings()

api_url = "https://10.23.0.1/restconf/data/ietf-interfaces:interfaces-
state/interface=GigabitEthernet1%2F0%2F23"

headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
           }

basicauth = ("admin", "cisco123!")

resp = requests.get(api_url, auth=basicauth, headers=headers, verify=False)
print(resp)
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Step 2: Examining the Code: Importing the required modules

The first three commands are used to import the required modules and to disable SSL certificate warnings.

```
import json
import requests
requests.packages.urllib3.disable_warnings()
```

A brief description of each command:

- **import json**: Imports the **json library** to let Python work with JSON data. It allows conversion between JSON text and Python objects like dictionaries and lists. (This will be explained in more detail later in this lab.)
- **import requests**: Imports the **requests library**, which lets Python send HTTP requests such as GET, POST, PUT, and DELETE.
- **requests.packages.urllib3.disable_warnings()**: When using the requests library, this hides SSL certificate warnings that appear when connecting to devices using self-signed certificates (common in lab environments).

Postman Equivalent: This is the Postman equivalent of disabling (toggle OFF) **"SSL certificate verification"** in Postman's Settings - it telling Python to *ignore SSL certificate warnings*.

Step 3: Examining the code: Creating the variables that will be the components of the request.

a. This command creates a variable named **api_url** and assigns the URL that will access the interface state information on D1's GigabitEthernet 1/0/23.

```
api_url = "https://10.23.0.1/restconf/data/ietf-interfaces:interfaces-
state/interface=GigabitEthernet1%2F0%2F23"
```

Postman Equivalent: This is Postman equivalent to using the **GET** request with the specific **URL** with the YANG model.

b. Create a dictionary variable named **headers** that has keys for **Accept** and **Content-type** and assign the keys the value **application/yang-data+json**.

```
headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
           }
```

Postman Equivalent: This is Postman equivalent to the key-value pairs you entered in the **Headers** tab, **Accept** and **Content-type**.

c. Create a Python tuple variable named **basicauth** that has two keys needed for authentication, **username** and **password**.

```
basicauth = ("cisco", "cisco123!")
```

Postman Equivalent: This is Postman equivalent to the information you added in the **Authorization** tab when selecting the **Basic Auth** method and the **username** and **password** key-value pairs.

Step 4: Examining the code: Creating a variable to send the request and store the JSON response.

This command sends the GET Request to the device:

```
resp = requests.get(api_url, auth=basicauth, headers=headers, verify=False)
```

Let's take a look at this part of the code.

In the previous step, we created several variables that define the components of our GET request:

- **api_url**: The **URL** of the RESTCONF resource we want to access.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

- **auth**: The **username** and **password** used for authentication.
- **headers**: The **content** format we want to send and receive (JSON).

Now, we used these variables as parameters in the **requests.get()** method to actually send the HTTP GET request to the RESTCONF API on D1. The **requests.get()** method combines all this information, sends the request, and waits for the device to reply.

We assigned the result to a variable named **resp**, which will store the device's response — including the HTTP status code and the returned YANG model in JSON format (if the request is successful).

Here's what each part of this command does:

- **requests.get()**: Actually sends the RESTCONF GET request to the device, just like clicking **Send** in Postman.
- **api_url**: Tells Python *where* to send the request (the RESTCONF URL you created earlier).
- **auth=basicauth**: Includes the username and password, just like the **Basic Auth** credentials you entered in Postman.
- **headers=headers**: Adds the **Accept** and **Content-Type** values that tell the device to send and receive JSON.
- **verify=False**: Ignores SSL certificate verification, just like turning off **SSL certificate verification** in Postman's settings.
- **resp**: Stores the device's response (including the HTTP status code and JSON data).

Step 5: Examining the code: Displaying the response code.

The last statement in the code prints the HTTP response code returned from the device.

```
print(resp)
```

Step 6: Run the code: Viewing the Status Code.

a. Save and run your code that you entered in Step 1.

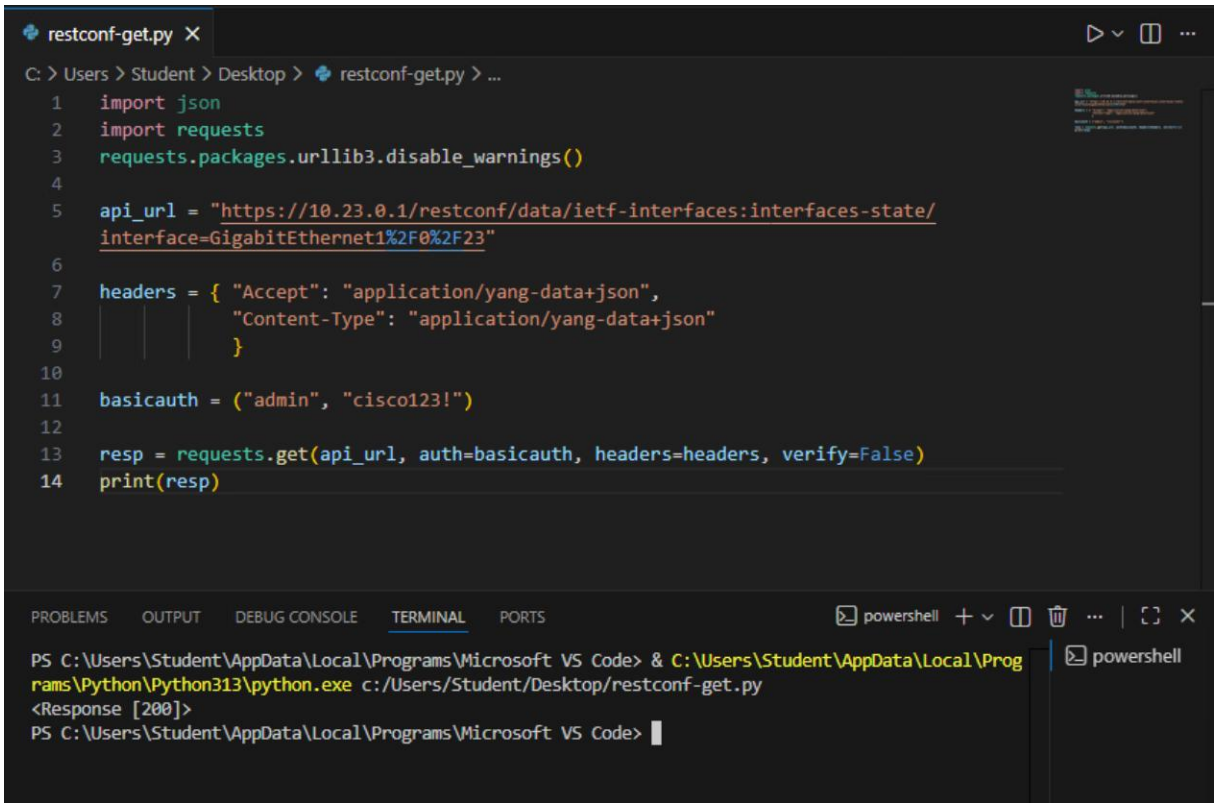
b. You should get the output shown below. If not, verify all previous steps in this part and ensure that SSH and RESTCONF are correctly configured on D1.

```
<Response [200]>
```

When you print **resp**, Python shows only a summary of the HTTP response - specifically the **status code** (200 = success). This is the same as seeing **Status: 200 OK** in Postman.

Here is an example shown in VS Code.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public



To view the actual data sent back from the device, you will need to access the **body** of the response using either **resp.text** (raw text) or **resp.json()** (structured JSON format), which will be covered in our next step.

Step 7: Format and display the JSON data received from D1.

a. Add the last two Python statements to your code:

```
import json
import requests
requests.packages.urllib3.disable_warnings()

api_url = "https://10.23.0.1/restconf/data/ietf-interfaces:interfaces-state/interface=GigabitEthernet1%2F0%2F23"

headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
          }

basicauth = ("cisco", "cisco123!")

resp = requests.get(api_url, auth=basicauth, headers=headers, verify=False)
print(resp)

response_json = resp.json()
print(response_json)
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

b. Let's examine the first new line of code.

```
response_json = resp.json()
```

When D1 sends its RESTCONF response, the data arrives in **JSON format**, which is the same type of structured text you saw in Postman under the **Body** > **Pretty** > **JSON** view.

However, in Python, **JSON** text is just a **long string of characters** until we convert it into **Python's** own **data structures** (like **dictionaries** and **lists**) that the program can easily work with.

This statement does that conversion.

The **.json()** method takes the **JSON text** from the RESTCONF response and **converts** it into a **Python** object (usually a **dictionary**).

Now, you can directly access parts of the data in Python, for example, interface names, IP addresses, or counters, just like working with any other Python variable.

Step 8: Run the code: Viewing the returned data and a closer look at JSON

a. Save and run your code that you entered in the previous step.

b. You should get output similar to the following.


```
{'ietf-interfaces:interface': {'name': 'GigabitEthernet1/0/23', 'type': 'iana-if-
type:ethernetCsmacd', 'admin-status': 'up', 'oper-status': 'lower-layer-down', 'last-
change': '2025-11-13T15:43:29.586+00:00', 'if-index': 31, 'phys-address':
'f8:e9:4f:9b:7f:e7, 'speed': '1000000000', 'statistics': {'discontinuity-time': '2025-11-
13T14:49:03.586+00:00', 'in-octets': '2257301', 'in-unicast-pkts': '20340', 'in-
broadcast-pkts': '622', 'in-multicast-pkts': '1176', 'in-discards': 0, 'in-errors': 0,
'in-unknown-protos': 0, 'out-octets': '1705206', 'out-unicast-pkts': '8698', 'out-
broadcast-pkts': '693', 'out-multicast-pkts': '2657', 'out-discards': 44909, 'out-
errors': 0}}}
```


c. When you print the variable **response_json**, you're seeing Python's version of the data that was returned by the device in **JSON format**.

JSON and **Python dictionaries** look very similar. They both store data in *key-value pairs*. But there are a few differences:

- **JSON** is text-based and stored as a continuous **string** (often called *serialized data*).
- **Python dictionaries** are an *in-memory data structure* that Python can directly use and manipulate.
- **JSON** always uses **double quotes** (" ") around keys and string values, while **Python** allows either **single** (' ') or **double** (" ") quotes.

  Note: JSON (the data format) always uses double quotes for keys and string values, as defined by its standard. When Python converts JSON into a dictionary and prints it, it usually displays the data using single quotes because that's just Python's default formatting.
- When Python data is converted to JSON (using **json.dumps()**), values like True, False, and None are automatically changed to JSON's lowercase true, false, and null to comply with the JSON standard required by RESTCONF and other APIs.

The **resp.json()** command you used in the previous step "deserialized" the JSON text (from the RESTCONF response) into a Python **dictionary**. This converted it from plain text (a **string**) into a structured object (a **dictionary**) that Python can easily work with. This is why your output now shows single quotes instead of double quotes used in JSON.

Note: **Serialization** and **deserialization** are how data moves between systems. When a device sends data (like RESTCONF responses), it *serializes* it into text (JSON). When Python receives that data, it *deserializes* it back into structured objects (like dictionaries) that your program can easily use.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Step 9: Format and display the returned data with json.dumps().

By default, when you print the variable **response_json**, Python displays the data all on one line. That's because **response_json** is now a **Python dictionary**, not JSON text. The original JSON text had no indentation.

Python dictionaries do not keep any formatting or indentation. They're structured by keys and braces, not whitespace. So Python just prints them in a compact, machine-friendly form.

a. To make the data easier to read, you can "pretty print" it using the **json.dumps()** function with the indent parameter. This converts the dictionary back into a formatted JSON-style string with indentation for readability.

```
print(json.dumps(response_json, indent=4))
```

b. Save and run your program. You should get the output shown below. This output is virtually identical to the output of your first Postman GET request.

Note: The following output is truncated to display only interfaces with a configuration.

```
<Response [200]>
{
    "ietf-interfaces:interfaces-state": {
        "interface": [
            {
                "name": "GigabitEthernet1/0/23",
                "type": "iana-if-type:ethernetCsmacd",
                <output omitted>
                "statistics": {
                    "discontinuity-time": "2025-11-13T14:49:03.586+00:00",
                    "in-octets": "2257301",
                    <output omitted>
                    "out-discards": 0,
                    "out-errors": 0
                }
            }
        ]
    }
}
```

When you call **json.dumps(response_json, indent=4)**, you are **converting** (or *serializing*) the Python dictionary **response_json** back into JSON-formatted text.

- **response_json** is a Python dictionary (the data already converted from JSON in the earlier step with **resp.json()**).
- The function **json.dumps()** takes that dictionary and turns it into a JSON string using JSON formatting rules (double quotes, colons, braces, etc.).
- The **indent=4** argument just makes it easier to read by adding indentation to each nested level of data.

So, when Python runs:

```
print(json.dumps(response_json, indent=4))
```

You are printing JSON text, not a dictionary. But the content is exactly the same, now with indentation.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Step 10: Format and display the returned data with pprint().

a. Another option is to use Python's built-in **pretty print** module, **pprint**. It works directly with Python objects (like dictionaries) and doesn't require converting them back to JSON.

To use it, you must first import it:

```
from pprint import pprint   # Put this near the top with other import commands
pprint(response_json)       # Add this to the end of your code
```

b. Save and run your program. You should get the output similar to what is shown below

You have now displayed the **same data** in three different ways:

- **print(response_json)**: Shows the raw, compact version of the **dictionary** on one line.
- **print(json.dumps(response_json, indent=4))**: Shows a formatted, **JSON**-style version, **string**, with indentation.
- **pprint(response_json)**: Shows a readable, Python-style version of the **dictionary** with indentation and line breaks.

Each method displays the same data; the difference is only in **format and readability**, not content.

```
<Response [200]>
{'ietf-interfaces:interface': {'name': 'GigabitEthernet1/0/23', 'type': 'iana-if-
type:ethernetCsmacd', 'admin-status': 'up', 'oper-status': 'lower-layer-down', 'last-
change': '2025-10-30T01:25:30.644+00:00', 'if-index': 31, 'phys-address':
'dc:0b:09:54:7f:97', 'speed': '1000000000', 'statistics': {'discontinuity-time':
'2025-06-19T21:14:00+00:00', 'in-octets': '3474878734', 'in-unicast-pkts': '3576834',
'in-broadcast-pkts': '4274', 'in-multicast-pkts': '2821', 'in-discards': 0, 'in-
errors': 0, 'in-unknown-protos': 0, 'out-octets': '649576024', 'out-unicast-pkts':
'16248118', 'out-broadcast-pkts': '281701', 'out-multicast-pkts': '5562386', 'out-
discards': 44909, 'out-errors': 0}}}
<class 'dict'>
{
    "ietf-interfaces:interface": {
        "name": "GigabitEthernet1/0/23",
        "type": "iana-if-type:ethernetCsmacd",
        "admin-status": "up",
        "oper-status": "lower-layer-down",
        "last-change": "2025-10-30T01:25:30.644+00:00",
        "if-index": 31,
        "phys-address": "dc:0b:09:54:7f:97",
        "speed": "1000000000",
        "statistics": {
            "discontinuity-time": "2025-06-19T21:14:00+00:00",
            "in-octets": "3474878734",
            "in-unicast-pkts": "3576834",
            "in-broadcast-pkts": "4274",
            "in-multicast-pkts": "2821",
            "in-discards": 0,
            "in-errors": 0,
            "in-unknown-protos": 0,
            "out-octets": "649576024",
            "out-unicast-pkts": "16248118",
            "out-broadcast-pkts": "281701",
            "out-multicast-pkts": "5562386",
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
            "out-discards": 44909,
            "out-errors": 0
        }
    }
}
{'ietf-interfaces:interface': {'admin-status': 'up',
                               'if-index': 31,
                               'last-change': '2025-10-30T01:25:30.644+00:00',
                               'name': 'GigabitEthernet1/0/23',
                               'oper-status': 'lower-layer-down',
                               'phys-address': 'dc:0b:09:54:7f:97',
                               'speed': '1000000000',
                               'statistics': {'discontinuity-time': '2025-06-
19T21:14:00+00:00',
                                              'in-broadcast-pkts': '4274',
                                              'in-discards': 0,
                                              'in-errors': 0,
                                              'in-multicast-pkts': '2821',
                                              'in-octets': '3474878734',
                                              'in-unicast-pkts': '3576834',
                                              'in-unknown-protos': 0,
                                              'out-broadcast-pkts': '281701',
                                              'out-discards': 44909,
                                              'out-errors': 0,
                                              'out-multicast-pkts': '5562386',
                                              'out-octets': '649576024',
                                              'out-unicast-pkts': '16248118'},
                               'type': 'iana-if-type:ethernetCsmacd'}}
```

Notice that **pprint** will display the data in a readable, structured format, although it may look slightly different from the JSON output produced by **json.dumps()**.

json.dumps() displays the data in JSON-style formatting (with double quotes and JSON syntax), while **pprint()** displays it in Python-style formatting (using single quotes and native Python object layout).

Part 8: Use a Python Script to Send a PUT Request

In this part, you will create a Python script to send a PUT request to D1. As was done in Postman, you will create a new loopback interface.

Step 1: Create the Python program to send PUT Request.

a. Open IDLE, VS Code or another Python IDE.

b. Create a new Python file and save the file as **restconf-put.py**.

c. Enter the following Python code:

```
import json
import requests
requests.packages.urllib3.disable_warnings()

api_url = "https://10.23.0.1/restconf/data/ietf-
interfaces:interfaces/interface=Loopback100"
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
           }

basicauth = ("admin", "cisco123!")

yangConfig = {
    "ietf-interfaces:interface": {
        "name": "Loopback100",
        "description": "[Student\'s Name] loopback interface",
        "type": "iana-if-type:softwareLoopback",
        "enabled": True,
        "ietf-ip:ipv4": {
            "address": [
                {
                    "ip": "10.100.0.1",
                    "netmask": "255.255.0.0"
                }
            ]
        },
        "ietf-ip:ipv6": {}
    }
}

resp = requests.put(api_url,
                    data=json.dumps(yangConfig),
                    auth=basicauth,
                    headers=headers,
                    verify=False)

if(resp.status_code >= 200 and resp.status_code <= 299):
    print("STATUS OK: {}".format(resp.status_code))
else:
    print('Error. Status Code: {} \nError message:
{}'.format(resp.status_code,resp.json()))
```

Step 2: Similar framework as GET Request.

This Python program uses the same overall framework as your earlier GET request.

The first few lines perform the same tasks you've already seen:

- **import json and requests**: Provides Python with the tools to handle JSON data and send HTTP requests.
- **requests.packages.urllib3.disable_warnings()**: Suppresses SSL certificate warnings when connecting to the device.
- **headers and basicauth**: Define the same RESTCONF headers and authentication credentials used previously.

These sections do not change between GET and PUT operations because they serve the same purpose — identifying who is sending the request and in what format the data is exchanged.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

In this program, there are **two key modifications** that we will examine next:

- The **api_url** now points to the specific interface you're creating or updating.
- The **resp** line uses the **requests.put()** method to send configuration data to the device instead of retrieving it.

You'll also notice new code that defines the interface configuration (the yangConfig dictionary), which represents the data being sent to the RESTCONF API.

Step 3: Update the API URL for the PUT request.

In this step, the **api_url** variable specifies the exact RESTCONF resource being configured. Unlike the previous GET request, which retrieved interface **state information**, this URL targets a specific interface **configuration** under the YANG model.

```
api_url = "https://10.23.0.1/restconf/data/ietf-
interfaces:interfaces/interface=Loopback100"
```

This path tells the device to apply configuration changes to the interface named **Loopback100**.

Because RESTCONF uses the YANG model's hierarchy, the resource name in the URL (interface=Loopback100) must exactly match the "name": "Loopback100" field in your JSON data.

Note: The resource path (ietf-interfaces:interfaces/interface=Loopback100) defines where in the YANG data tree the configuration will be created or modified. In this case, within the *interfaces* container of the *ietf-interfaces* model.

Step 4: Update the PUT request to configure the device.

In this step, the **requests.put()** method is used instead of **requests.get()**. While GET requests retrieve data from the device, a **PUT** request is used to **create or replace configuration data** on the device.

```
resp = requests.put(api_url,
                    data=json.dumps(yangConfig),
                    auth=basicauth,
                    headers=headers,
                    verify=False)
```

We assigned the result to a variable named resp, which will store the device's response, including the HTTP status code and any message returned by the RESTCONF server.

If the request is successful, the device will apply the configuration contained in **yangConfig** to the interface specified by **api_url**. Just as with the GET request, this single line performs all the work of sending the RESTCONF request. The key difference is that **requests.put()** sends configuration data rather than retrieving it.

This step is nearly identical to the GET request you used earlier, with two key differences:

1. **requests.put()**: Instead of retrieving data (as with **requests.get()**), this **sends** data to the device to create or update configuration.
2. **data=json.dumps(yangConfig)**: This converts the Python dictionary **yangConfig** into properly formatted JSON text before sending it in the request body.

The configuration data being sent to the device is stored in the variable **yangConfig**, which contains the interface settings formatted as a Python dictionary. This dictionary follows the structure defined by the YANG data model. We'll look at its contents more closely in the next step.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Step 5: Create the Configuration Dictionary.

The **yangConfig** variable is a Python dictionary that defines the interface configuration you want to send to the device. It follows the structure of the **YANG data model**, which describes how network configuration data is organized.

Each key in the dictionary corresponds to an element in the YANG model (for example, interface name, type, or IP address). This makes it easy for RESTCONF to understand what configuration changes to apply.

```
yangConfig = {
    "ietf-interfaces:interface": {
        "name": "Loopback100",
        "description": "[Student\'s Name] loopback interface",
        "type": "iana-if-type:softwareLoopback",
        "enabled": True,
        "ietf-ip:ipv4": {
            "address": [
                {
                    "ip": "10.10.100.1",
                    "netmask": "255.255.0.0"
                }
            ]
        },
        "ietf-ip:ipv6": {}
    }
}
```

Here, you are defining a **Loopback100** interface, its description, type (softwareLoopback), and IP settings. When this dictionary is later sent to the device, it will be converted (serialized) into JSON text so the RESTCONF API can process it.

Step 6: Sending the Configuration to the Device.

When your program runs, the following line sends the configuration to the device through RESTCONF:

```
resp = requests.put(api_url,
                    data=json.dumps(yangConfig),
                    auth=basicauth,
                    headers=headers,
                    verify=False)
```

Here's how this works:

- **requests.put()**: Sends an HTTP PUT request to the device at the RESTCONF URL you defined earlier.
- **data=json.dumps(yangConfig)**: Converts (serializes) the Python dictionary **yangConfig** into JSON-formatted text before it is sent. Devices expect RESTCONF configuration data in JSON format, not in Python structures.
- **yangConfig**: Holds the configuration details (the loopback interface) as a Python dictionary following the YANG data model.
- **resp**: Stores the device's response to your request. This includes the HTTP status code (for example 200 = OK or 201 = Created) and any returned messages.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

In short, this single line takes a Python dictionary, turns it into JSON text, sends it to the device using RESTCONF, and saves the device's reply for you to check.

Step 7: The matching interface name

When creating or modifying interfaces using RESTCONF, the interface name must follow specific rules and **must match exactly** in both the URL and the JSON body.

```
api_url = "https://10.23.0.1/restconf/data/ietf-
interfaces:interfaces/interface=Loopback100"


yangConfig = {
    "ietf-interfaces:interface": {
         "name": "Loopback100",
         <rest of dictionary omitted>
```

Key Rules:

- The name in the URL and the "name" in the JSON body **must be identical**.
    - **URL**: .../interface=Loopback100
    - **JSON**: "name": "Loopback100"
- Interface names are **case-sensitive** (Loopback100 ≠ loopback100).
- **No spaces** are allowed in interface names.
- The interface name must match the **actual IOS XE interface name** (as shown in **show running-config** or **show ip interface brief**).

If these values do not match exactly, the device will return a **400 Bad Request** error because the RESTCONF path and the YANG model key are mismatched.

Step 8: Run the code: Using PUT to configure an interface.

a. Run the program to send the PUT request to D1. If successful, the terminal should display:

```
STATUS OK: 201
```

The **201 Created** response means the device successfully received the configuration and created the new interface resource defined in your program. If you receive a different code (such as 400 or 403), check your code syntax, the interface **name** (case-sensitive), and that RESTCONF is properly configured on D1.

b. You can verify that the interface was created by entering **show ip interface brief** on D1. You should see output similar to the following:

```
D1# show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/1   unassigned      YES unset  administratively down down
<output omitted>
Loopback100            10.10.100.1     YES other  up                    up
```

The **Method** column shows how the interface's IP address was assigned. When it displays **"other"**, it means the configuration was applied through an external method. In this case, **RESTCONF** was used, not manually entered in the CLI or obtained via DHCP.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Part 9: Real-World Example — Using Python and RESTCONF for Interface Monitoring

In this final section, you'll see how Python and RESTCONF can be combined to create a simple but powerful network monitoring tool.

This example retrieves live interface statistics from the device using the **ietf-interfaces:interfaces-state** YANG model and formats the results into a clean, readable table.

```
                                    Counters reset:  06/19/2025 21:14:00
                                Input (I) and Output (O) Interface Traffic

+-------------------------+-------------------+-------------+---------------+-----------------+---------------------+---------------+-----------------+---------------------+
|                Interface |       Oper Status | Speed (b/s) |       I-Bytes | I-Unicast Packets | I-Broadcast Packets |       O-Bytes | O-Unicast Packets | O-Broadcast Packets |
+-------------------------+-------------------+-------------+---------------+-----------------+---------------------+---------------+-----------------+---------------------+
|      GigabitEthernet0/0 |              down |      1 Gb/s |         2,080 |              32 |                   0 |         1,798 |              21 |                   0 |
|    GigabitEthernet1/0/1 | lower-layer-down |      1 Gb/s |   372,842,094 |         873,452 |               1,668 | 1,297,954,834 |       4,388,154 |                 370 |
|   GigabitEthernet1/0/10 | lower-layer-down |     10 Mb/s |42,533,768,096 |      61,836,236 |              25,437 | 1,869,640,843 |      44,700,856 |             136,983 |
|   GigabitEthernet1/0/11 |                up |     10 Mb/s | 1,008,526,330 |       1,440,926 |               4,604 |   778,867,385 |      13,778,545 |             264,823 |
|   GigabitEthernet1/0/12 |                up |     10 Mb/s | 2,802,055,982 |       2,617,887 |               8,667 |   984,603,941 |      14,070,868 |             267,991 |
|   GigabitEthernet1/0/13 |                up |     10 Mb/s | 4,518,618,350 |       4,312,784 |              18,674 | 1,868,316,499 |      18,802,548 |             291,368 |
|   GigabitEthernet1/0/14 |                up |     10 Mb/s | 1,942,659,588 |       2,191,143 |               5,021 | 1,561,557,876 |      15,791,559 |             314,412 |
|   GigabitEthernet1/0/15 |                up |     10 Mb/s | 3,202,108,316 |       3,972,561 |              20,160 | 2,969,328,168 |      21,465,835 |             211,221 |
|   GigabitEthernet1/0/16 |                up |     10 Mb/s | 2,275,255,603 |       2,449,058 |               4,170 | 2,411,450,469 |      15,493,147 |             297,149 |
|   GigabitEthernet1/0/17 |                up |     10 Mb/s | 2,180,223,641 |       2,223,059 |               6,829 | 1,375,542,721 |      14,234,352 |             261,185 |
|   GigabitEthernet1/0/18 |                up |     10 Mb/s | 1,312,572,834 |       1,537,301 |               3,063 |   836,967,661 |      15,218,104 |             315,619 |
|   GigabitEthernet1/0/19 |                up |     10 Mb/s | 5,860,471,587 |       4,752,281 |               5,693 | 1,493,722,943 |      15,309,203 |             219,185 |
|    GigabitEthernet1/0/2 | lower-layer-down |      1 Gb/s |           136 |               2 |                   0 |            68 |               1 |                   0 |
|   GigabitEthernet1/0/20 |                up |     10 Mb/s | 3,257,124,832 |       2,950,620 |               5,286 | 1,063,349,052 |      13,876,916 |             207,338 |
|   GigabitEthernet1/0/21 |                up |     10 Mb/s | 5,688,012,918 |       4,755,508 |              13,534 | 2,268,503,168 |      15,469,977 |             278,161 |
```

The following Python program demonstrates how to collect and format interface statistics using RESTCONF. If you are new to Python, you'll see a few new concepts here, but the comments will help guide you through what each part of the program does and how it all works together.

```
# ==================== restconf_interface-state_table.py ==================== #

from getpass import getpass
from prettytable import PrettyTable
from datetime import datetime
import requests
requests.packages.urllib3.disable_warnings()

#################################################################
###############  Username and Password ##########################

# Prompts and returns SSH username and password (not shown)
def get_priv15_credentials():
    # Username entered is shown in clear text
    username = input('Enter privilege level 15 username: ')

    # Password entered is not shown
    password = getpass('Enter privilege level 15 password: ')

    return username, password

# Collect credentials (privilege 15 recommended for full access)
username, password = get_priv15_credentials()

print("\nPlease wait... this could take a minute or two...\n")

#################################################################
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
##################  RESTCONF API call ##########################

api_url = "https://[2607:f380:80f:f010::2]/restconf/data/ietf-interfaces:interfaces-
state"

# RESTCONF request headers (request/response using YANG-JSON format)
headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
           }

basicauth = (username, password)

# Send GET request to retrieve operational interface data
response = requests.get(api_url, auth=basicauth, headers=headers, verify=False)

# Basic status check (optional but helpful)
if response.status_code != 200:
    print(f"Error {response.status_code}: {response.text}")
    exit(1)

################################################################
########  Convert JSON Object to Python Dictionary #############

# Convert JSON response body into a Python dictionary
response_json = response.json()   # From object to dictionary

################################################################
################  Parsing Python dictionary ###############

# List to store the per-interface dictionaries from the response
interface_list = []  # type list


# Create new working variable (dictionary) with respsonse_json key-value pairs
restconf_interface_state = response_json  # type(s) dictionary

# Extract the portion of the data that contains interface information
for entry in restconf_interface_state: # Iterates through the top-level dictionary
    # This key holds the operational interface information (as a Python dictionary)
    interface_state = restconf_interface_state["ietf-interfaces:interfaces-state"]
    # Inside that dictionary, "interface" is a list where each item represents one
interface
    interface_list = interface_state["interface"]

################  Formatting Discontinuity Time ###############

# Capture the "discontinuity-time" (when counters last reset)
# ISO 8601 datetime string
iso_datetime = " "
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
for entry in interface_list:
    iso_datetime = entry['statistics']['discontinuity-time']
    if iso_datetime != " ":
        break

# Convert the ISO 8601 timestamp string to a Python datetime object
dt_object = datetime.fromisoformat(iso_datetime)

# # Reformat Python datetime object into a more readable style
formatted_datetime = dt_object.strftime("%m/%d/%Y %H:%M:%S")

################ Pretty Table ###############

# Initialize PrettyTable for clean, aligned tabular output
# Requires 'prettytable' package: pip install prettytable
table = PrettyTable()

print("\n")
print(" " *62,"Counters reset: ", formatted_datetime)
print("Input (I) and Output (O) Interface Traffic".center(160),"\n")

# Define the column headers shown in the output table
table.field_names = ["Interface", "Oper Status", "Speed (b/s)", "I-Bytes", "I-Unicast
Packets", "I-Broadcast Packets", "O-Bytes", "O-Unicast Packets", "O-Broadcast
Packets"]

################ Iterate through list of interfaces ###############

# Loop through each interface dictionary and format data for the table
# Each entry in the interface_list is a dictionary representing an individual
interface
for entry in interface_list:
    # Example debugging prints (optional for learners)
    # print(entry['name'], entry['oper-status'])
    # print(entry['statistics']['in-errors'])

    table.align = "r"  # Right-align numbers for better readability

    # Convert interface speed (bps) into human-friendly units
    int_speed = int(entry['speed'])
    if int_speed == 10000000:
        formatted_speed = '10 Mb/s'
    elif int_speed == 100000000:
        formatted_speed = '100 Mb/s'
    if int_speed == 1000000000:
        formatted_speed = '1 Gb/s'
    if int_speed == 10000000000:
        formatted_speed = '10 Gb/s'

    # Convert counters from string → int → formatted string with commas
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
        int_in_octets = int(entry['statistics']['in-octets'])
        formatted_int_in_octets = "{:,}".format(int_in_octets)

        int_in_unicast_pkts = int(entry['statistics']['in-unicast-pkts'])
        formatted_in_unicast_pkts = "{:,}".format(int_in_unicast_pkts)

        int_in_broadcast_pkts = int(entry['statistics']['in-broadcast-pkts'])
        formatted_int_in_broadcast_pkts = "{:,}".format(int_in_broadcast_pkts)

        int_out_octets = int(entry['statistics']['out-octets'])
        formatted_int_out_octets = "{:,}".format(int_out_octets)

        int_out_unicast_pkts = int(entry['statistics']['out-unicast-pkts'])
        formatted_out_unicast_pkts = "{:,}".format(int_out_unicast_pkts)

        int_out_broadcast_pkts = int(entry['statistics']['out-broadcast-pkts'])
        formatted_int_out_broadcast_pkts = "{:,}".format(int_out_broadcast_pkts)

        # Add one row to the table for this interface
        table.add_row([
            entry['name'],
            entry['oper-status'],
            formatted_speed,
            formatted_int_in_octets,
            formatted_in_unicast_pkts,
            formatted_int_in_broadcast_pkts,
            formatted_int_out_octets,
            formatted_out_unicast_pkts,
            formatted_int_out_broadcast_pkts
        ])

    ################  Print the Table ###############

    # Print the completed table
    print(table)
```

This example demonstrates the power of structured data and how device management tools like RESTCONF, combined with programming languages such as Python, can make that data both accessible and meaningful. As you develop your Python skills, you'll be able to expand this approach — gathering information from multiple YANG models, analyzing trends, generating reports, and even automating network configuration tasks across many devices.

Programs Used in this Lab

The following Python scripts were used in this lab:

```
# ========================= restconf-get.py ========================= #
import json
import requests
requests.packages.urllib3.disable_warnings()
from pprint import pprint
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
api_url = "https://10.23.0.1/restconf/data/ietf-interfaces:interfaces"

headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
            }

basicauth = ("cisco", "cisco123!")

resp = requests.get(api_url, auth=basicauth, headers=headers, verify=False)
print(resp)

response_json = resp.json()

print(response_json)
print(json.dumps(response_json, indent=4))
pprint(response_json)


# ========================= restconf-put.py ========================= #
import json
import requests
requests.packages.urllib3.disable_warnings()

api_url = "https://10.23.0.1/restconf/data/ietf-
interfaces:interfaces/interface=Loopback100"

headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
           }
basicauth = ("cisco", "cisco123!")

yangConfig = {
    "ietf-interfaces:interface": {
        "name": "Loopback100",
        "description": "[Student\'s Name] loopback interface",
        "type": "iana-if-type:softwareLoopback",
        "enabled": True,
        "ietf-ip:ipv4": {
            "address": [
                {
                    "ip": "10.10.100.1",
                    "netmask": "255.255.255.0"
                }
            ]
        },
        "ietf-ip:ipv6": {}
    }
}
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
resp = requests.put(api_url, data=json.dumps(yangConfig), auth=basicauth,
headers=headers, verify=False)

if(resp.status_code >= 200 and resp.status_code <= 299):
    print("STATUS OK: {}".format(resp.status_code))
else:
    print('Error. Status Code: {} \nError message: {}'.format(resp.status_code,
resp.json()))


# ================== restconf_interface-state_table.py ================== #

from getpass import getpass
from prettytable import PrettyTable
from datetime import datetime
import requests
requests.packages.urllib3.disable_warnings()

#####################################################################
##############  Username and Password ##########################

# Prompts and returns SSH username and password (not shown)
def get_priv15_credentials():
    # Username entered is shown in clear text
    username = input('Enter privilege level 15 username: ')

    # Password entered is not shown
    password = getpass('Enter privilege level 15 password: ')

    return username, password


# Collect credentials (privilege 15 recommended for full access)
username, password = get_priv15_credentials()

print("\nPlease wait... this could take a minute or two...\n")

#####################################################################
##################  RESTCONF API call ##########################

api_url = "https://[2607:f380:80f:f010::2]/restconf/data/ietf-interfaces:interfaces-
state"

# RESTCONF request headers (request/response using YANG-JSON format)
headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
          }

basicauth = (username, password)
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
# Send GET request to retrieve operational interface data
response = requests.get(api_url, auth=basicauth, headers=headers, verify=False)

# Basic status check (optional but helpful)
if response.status_code != 200:
    print("Error {response.status_code}: {response.text}")
    exit(1)

################################################################
########  Convert JSON Object to Python Dictionary #############

# Convert JSON response body into a Python dictionary
response_json = response.json()    # From object to dictionary

################################################################
################  Parsing Python dictionary ###############

# List to store the per-interface dictionaries from the response
interface_list = []  # type list

# Create new working variable (dictionary) with respsonse_json key-value pairs
restconf_interface_state = response_json  # type(s) dictionary

# Extract the portion of the data that contains interface information
for entry in restconf_interface_state: # Iterates through the top-level dictionary
    # This key holds the operational interface information (as a Python dictionary)
    interface_state = restconf_interface_state["ietf-interfaces:interfaces-state"]
    # Inside that dictionary, "interface" is a list where each item represents one
interface
    interface_list = interface_state["interface"]

################  Formatting Discontinuity Time ###############

# Capture the "discontinuity-time" (when counters last reset)
# ISO 8601 datetime string
iso_datetime = " "

for entry in interface_list:
    iso_datetime = entry['statistics']['discontinuity-time']
    if iso_datetime != " ":
        break

# Convert the ISO 8601 timestamp string to a Python datetime object
dt_object = datetime.fromisoformat(iso_datetime)

# # Reformat Python datetime object into a more readable style
formatted_datetime = dt_object.strftime("%m/%d/%Y %H:%M:%S")

################ Pretty Table ###############
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
# Initialize PrettyTable for clean, aligned tabular output
# Requires 'prettytable' package: pip install prettytable
table = PrettyTable()

print("\n")
print(" " *62,"Counters reset: ", formatted_datetime)
print("Input (I) and Output (O) Interface Traffic".center(160),"\n")

# Define the column headers shown in the output table
table.field_names = ["Interface", "Oper Status", "Speed (b/s)", "I-Bytes", "I-Unicast
Packets", "I-Broadcast Packets", "O-Bytes", "O-Unicast Packets", "O-Broadcast
Packets"]

################ Iterate through list of interfaces ###############

# Loop through each interface dictionary and format data for the table
# Each entry in the interface_list is a dictionary representing an individual
interface
for entry in interface_list:
    # Example debugging prints (optional for learners)
    # print(entry['name'], entry['oper-status'])
    # print(entry['statistics']['in-errors'])

    table.align = "r"  # Right-align numbers for better readability

    # Convert interface speed (bps) into human-friendly units
    int_speed = int(entry['speed'])
    if int_speed == 10000000:
        formatted_speed = '10 Mb/s'
    elif int_speed == 100000000:
        formatted_speed = '100 Mb/s'
    if int_speed == 1000000000:
        formatted_speed = '1 Gb/s'
    if int_speed == 10000000000:
        formatted_speed = '10 Gb/s'

    # Convert counters from string → int → formatted string with commas
    int_in_octets = int(entry['statistics']['in-octets'])
    formatted_int_in_octets = "{:,}".format(int_in_octets)

    int_in_unicast_pkts = int(entry['statistics']['in-unicast-pkts'])
    formatted_in_unicast_pkts = "{:,}".format(int_in_unicast_pkts)

    int_in_broadcast_pkts = int(entry['statistics']['in-broadcast-pkts'])
    formatted_int_in_broadcast_pkts = "{:,}".format(int_in_broadcast_pkts)

    int_out_octets = int(entry['statistics']['out-octets'])
    formatted_int_out_octets = "{:,}".format(int_out_octets)
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
        int_out_unicast_pkts = int(entry['statistics']['out-unicast-pkts'])
        formatted_out_unicast_pkts = "{:,}".format(int_out_unicast_pkts)

        int_out_broadcast_pkts = int(entry['statistics']['out-broadcast-pkts'])
        formatted_int_out_broadcast_pkts = "{:,}".format(int_out_broadcast_pkts)

        # Add one row to the table for this interface
        table.add_row([
            entry['name'],
            entry['oper-status'],
            formatted_speed,
            formatted_int_in_octets,
            formatted_in_unicast_pkts,
            formatted_int_in_broadcast_pkts,
            formatted_int_out_octets,
            formatted_out_unicast_pkts,
            formatted_int_out_broadcast_pkts
        ])

################  Print the Table ###############

# Print the completed table
print(table)
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public